UFCD 3935 João Monge

UFCD 3935 – Programação C#

Aluno: Manoel Silva Barata

1. Introdução

Este documento descreve o processo de criação de uma aplicação móvel utilizando .NET MAUI, desenvolvida no Visual Studio, com persistência de dados através de uma base de dados SQLite local. A aplicação terá como objetivo permitir:

• A inserção de dados através de um formulário;

• A visualização dos dados armazenados numa base de dados local.

Todo o desenvolvimento é realizado sem recurso à consola, utilizando exclusivamente as funcionalidades gráficas do Visual Studio.

2. Objetivos de Aprendizagem

No final deste trabalho, o aluno deverá ser capaz de:

• Criar um projeto .NET MAUI no Visual Studio;

• Integrar uma base de dados SQLite numa aplicação móvel;

• Criar páginas de interface gráfica (UI);

• Inserir e ler dados de uma base de dados local;

• Compreender a separação entre interface, dados e lógica da aplicação.

3. Criação do Projeto

1. Abrir o Visual Studio 2022.

2. Selecionar a opção Create a new project.

3. Escolher o template .NET MAUI App.

4. Atribuir o nome ao projeto (exemplo: Ex9_Pessoas_App).

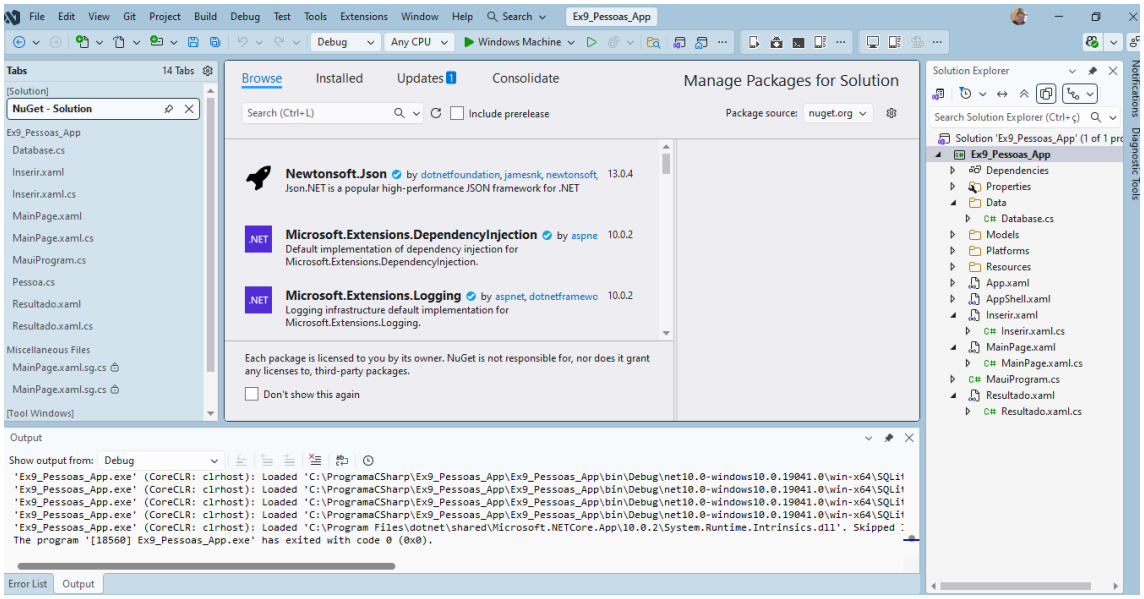
5. Concluir a criação do projeto.

Após este passo, a estrutura base da aplicação será automaticamente criada.

4. Integração da Base de Dados SQLite

Para permitir o armazenamento local de dados, será utilizada uma base de dados SQLite.

4.1 Instalação da biblioteca



1. No Solution Explorer, clicar com o botão direito sobre o projeto.

2. Selecionar Manage NuGet Packages.

3. Procurar e instalar o pacote:

o sqlite-net-pcl

Esta biblioteca simplifica a criação e gestão de bases de dados SQLite em aplicações MAUI.

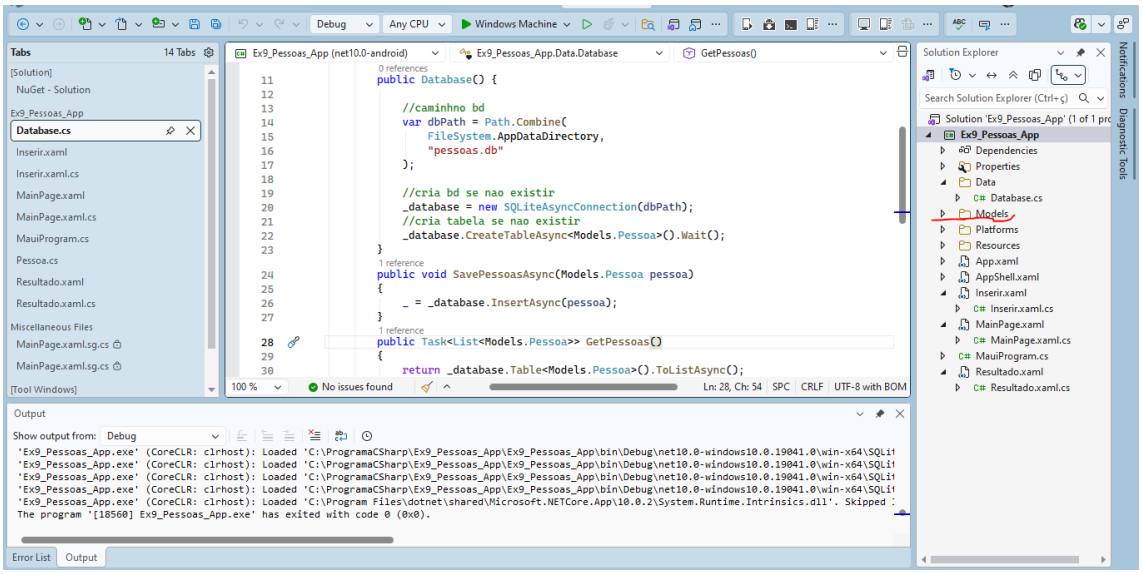
5. Modelo de Dados

A aplicação irá armazenar informação relativa a pessoas.

Cada registo deverá conter:

- Um identificador único;
- Um nome;
- Uma idade.

O modelo de dados representa a estrutura da tabela na base de dados e deve ser criado numa pasta denominada Models.



Código Modelo de Dados – Pessoas.cs (folder: Models):

```
using SQLite;
using System;
using System.Collections.Generic;
using System.Text;

namespace Ex9_Pessoas_App.Models
{
    internal class Pessoa
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
        [NotNull]
        public string Nome { get; set; }
        [NotNull]
        public int Idade { get; set; }


    }
}
```

6. Base de Dados

Será criada uma classe responsável por:

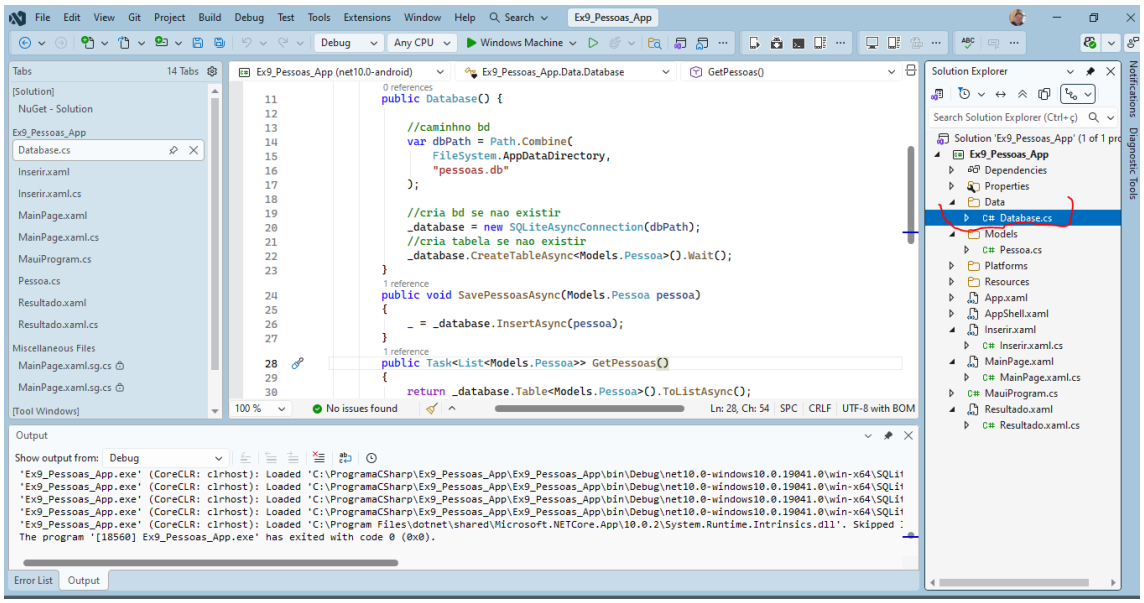
• Criar a base de dados local;

• Criar a tabela correspondente ao modelo;

• Inserir dados;

• Ler dados armazenados.

Esta classe deve ficar isolada numa pasta própria (ex.: Data) e será utilizada pelas páginas da aplicação.



Código Base de Dados – Database.cs (folder: Data)

```
using SQLite;
using System;
using System.Collections.Generic;
using System.Text;

namespace Ex9_Pessoas_App.Data
{
    internal class Database
    {
        private readonly SQLiteAsyncConnection _database;
        public Database() {

            //caminhno bd
            var dbPath = Path.Combine(
                FileSystem.AppDataDirectory,
                "pessoas.db"
            );

            //cria bd se nao existir
            _database = new SQLiteAsyncConnection(dbPath);
            //cria tabela se nao existir
            _database.CreateTableAsync<Models.Pessoa>().Wait();
        }
        public void SavePessoasAsync(Models.Pessoa pessoa)
        {
            _ = _database.InsertAsync(pessoa);
        }
        public Task<List<Models.Pessoa>> GetPessoas()
        {
            return _database.Table<Models.Pessoa>().ToListAsync();
        }

    }
}
```

7. Página de Inserção de Dados

A primeira página da aplicação deverá conter um formulário com:

Aluno: Manoel Silva Barata - 25481 – Programador de Informática

• Um campo de texto para o nome;

• Um campo numérico para a idade;

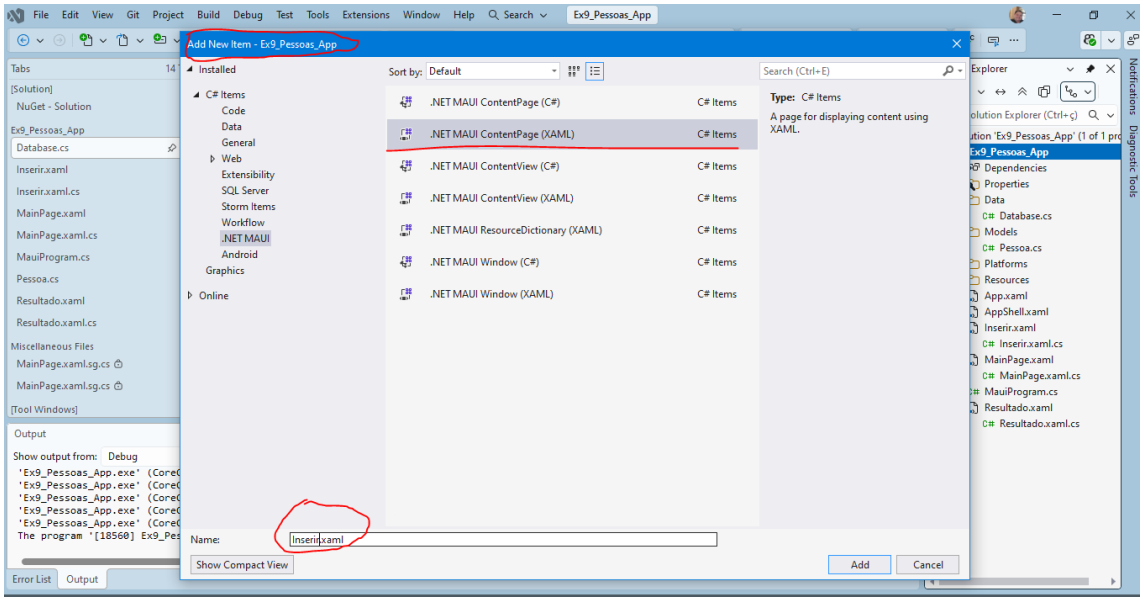
• Um botão para guardar os dados.

Funcionamento esperado:

• O utilizador preenche o formulário;

• Ao clicar no botão, os dados são guardados na base de dados SQLite.

ADD New Item – NETMAUI ContentPage – Inserir.xaml



Códigos – dois arquivos para a Função Inserir (folder: Ex9_Pessoas_APP)

a) **Interface gráfica: Inserir.xaml**

```
  <?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Ex9_Pessoas_App.Inserir"
             Title="Inserir novo colaborador">
    <VerticalStackLayout>
        <Label Text="Nome:" />
        <Entry x:Name="Nome" Placeholder="Insira o seu nome" />
        <Label Text="Idade:" />
        <Entry x:Name="Idade" Placeholder="Insira a sua idade" />
        <Button Text="Inserir" Clicked="InserirPessoa" />
        <Button Text="Voltar" Clicked="Voltar" />
    </VerticalStackLayout>
</ContentPage>
```

b) **Programação em C# - Inserir.xaml.cs**

```
using Ex9_Pessoas_App.Data;

namespace Ex9_Pessoas_App;

public partial class Inserir : ContentPage
{
```

```
    Database _db;
    public Inserir()
      {
            InitializeComponent();
      }

    private void InserirPessoa(object sender, EventArgs e)
    {
        var pessoa = new Models.Pessoa
        {
            Nome = Nome.Text,
            Idade = int.Parse(Idade.Text)
        };

        _db =
App.Current.Handler.MauiContext.Services.GetService<Database>();
        _db.SavePessoasAsync(pessoa);

    }

    private void Voltar(object sender, EventArgs e)
    {
    Navigation.PopAsync();
    }
}
```

8. Página de Visualização de Dados

ADD New Item – NETMAUI ContentPage – Resultado.xaml

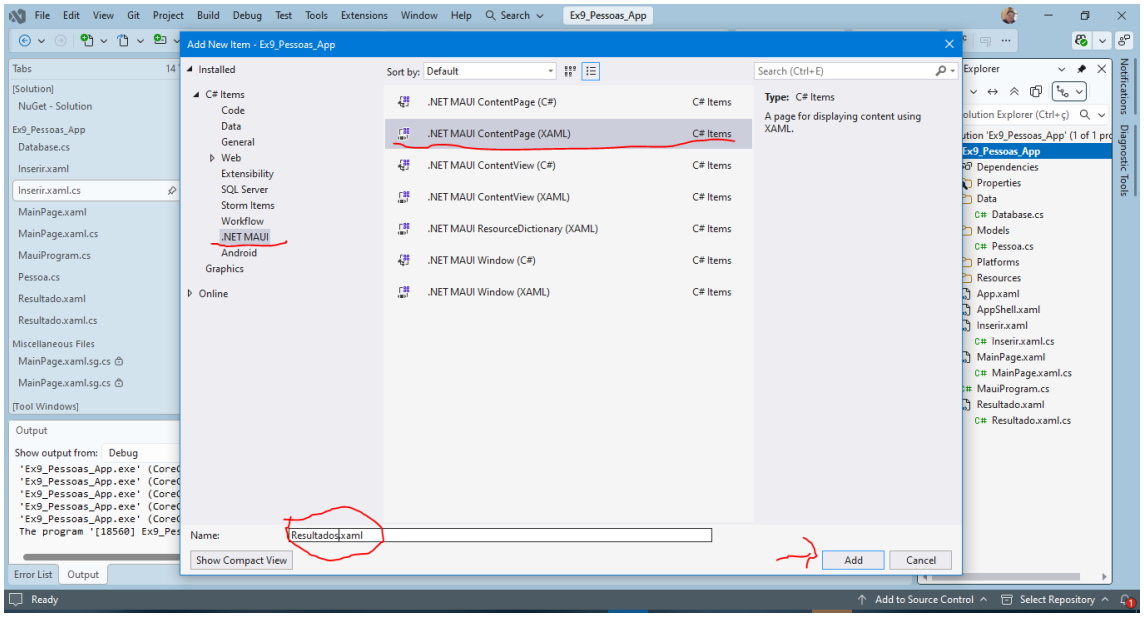
A segunda página da aplicação deverá apresentar:

• Uma lista com os registos existentes na base de dados.

Funcionamento esperado:

• Sempre que a página é aberta, os dados são lidos da base de dados;

• Cada item da lista apresenta a informação guardada (ex.: nome e idade).



Códigos – dois arquivos para a Função Resultado (folder: Ex9_Pessoas_APP)

a) Interface gráfica: Resultado.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Ex9_Pessoas_App.Resultado"
             Title="Colaboradores cadastrados:">
    <VerticalStackLayout>
        <CollectionView
        x:Name="PessoasCollectionView"
        ItemsSource="{Binding Pessoas}">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <HorizontalStackLayout  Padding="10">
                        <Label FontSize="16">
                            <Label.Text>
                                <Binding Path="Nome" StringFormat="O
colaborador de nome {0} " />
                            </Label.Text>
                        </Label>

                        <Label FontSize="16">
                            <Label.Text>
                                <Binding Path="Idade" StringFormat=" possui
idade de {0} anos" />
                            </Label.Text>
                        </Label>
                    </HorizontalStackLayout>
            </DataTemplate>
            </CollectionView.ItemTemplate>
            </CollectionView>

    <VerticalStackLayout>
    <Button
        x:Name="VoltarBtn"
        Text="Clique aqui para voltar"
        Clicked="Voltar"
        HorizontalOptions="Fill" />
      </VerticalStackLayout>

    </VerticalStackLayout>
</ContentPage>
```

b) Programação em C# - Resultado.xaml.cs

```
namespace Ex9_Pessoas_App;

public partial class Resultado : ContentPage
{
    public Resultado()
    {
        InitializeComponent();
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
```

```
        var db =
App.Current.Handler.MauiContext.Services.GetService<Data.Database>();
        var pessoas = await db.GetPessoas();

        // Define diretamente o ItemsSource do CollectionView
        PessoasCollectionView.ItemsSource = pessoas;
    }
    //Botão voltar para a página principal

    private void Voltar(object sender, EventArgs e)
    {
        Navigation.PushAsync(new Inserir());

    }

}
```

9. Navegação

A aplicação deverá permitir a navegação entre as duas páginas, podendo ser utilizada:

• Navegação por botões.

O objetivo é permitir ao utilizador alternar facilmente entre:

• Inserção de dados;

• Visualização de dados.

Códigos – dois arquivos para a página principal da Aplicação - responsável pelo início e pela navegação na aplicação - (folder: Ex9_Pessoas_APP)

a) Interface gráfica: MainPage.xaml

```
      <?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Ex9_Pessoas_App.MainPage">

    <ScrollView>
        <VerticalStackLayout
            Padding="30,0"
            Spacing="25">
            <Image
                Source="dotnet_bot.png"
                HeightRequest="185"
                Aspect="AspectFit"
                SemanticProperties.Description="dot net bot in a submarine
number ten" />

             <Label
                Text="Formulário - Cadastro de Colaboradores"
                Style="{StaticResource SubHeadline}"
                SemanticProperties.HeadingLevel="Level2"
                SemanticProperties.Description="Welcome to dot net Multi
platform App U I" />


            <Button
            x:Name="InserirBtn"
```

```
            Text="Clique aqui para inserir"
             Clicked="InserirPessoa"
             HorizontalOptions="Fill" />

            <Button
            x:Name="ResultadoBtn"
            Text="Clique aqui para resultado"
            Clicked="MostrarResultado"
            HorizontalOptions="Fill" />


        </VerticalStackLayout>
    </ScrollView>

</ContentPage>
```

b) Programação em C#: MainPage.xaml.cs

```
namespace Ex9_Pessoas_App
{
    public partial class MainPage : ContentPage
    {
        int count = 0;

        public MainPage()
        {
            InitializeComponent();
        }


        //Botão Inserir Pessoa
        private void InserirPessoa(object sender, EventArgs e)
        {
            Navigation.PushAsync(new Inserir());
        }

        //Botão Listar Pessoas
        private void MostrarResultado(object sender, EventArgs e)
        {
            Navigation.PushAsync(new Resultado());

        }

    }
}
```

c) Com relação a navegação também foi inserido um botão de voltar na página do resultado para retornar a página de Inserir:

- **Interface Gráfica do Botão voltar – página resultado.xaml**

```
  <VerticalStackLayout>
  <Button
      x:Name="VoltarBtn"
      Text="Clique aqui para voltar"
      Clicked="Voltar"
      HorizontalOptions="Fill" />
          </VerticalStackLayout>
```

- **Programação em C# do Botão voltar – página resultado.xaml.cs**

```
//Botão voltar para a página Inserir

 private void Voltar(object sender, EventArgs e)
 {
     Navigation.PushAsync(new Inserir());

    }
```